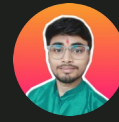




Save



Ansh Mittal

Currying



It's **way** to call **inner functions** where you can pass arguments **partially** or pass multiple arguments in a function but **1 argument at a time** .

```
Currying

function sum(a) {
  return function (b) {
    return a + b;
  };
}

const answer = sum(5)(6);
console.log(answer); // 11
```



What is the practical use of currying

Here we want to **calculate Discounted price** of product but we have a **multiple discount** and **price** is **one**. so we have to call this function **partially**. **First** for assign **price** and **Second** for passing **discount** percentage and **calculate** the discounted Price. If we have **multiple discount** then we can call **again** only inner function which is responsible to calculate discounted Price.



Currying

```
function calculateDiscountAmount(price) {  
  return function (discountRate) {  
    return price * discountRate;  
  }  
}  
  
const discountAmount = calculateDiscountAmount(300);  
  
console.log(discountAmount(0.5)); // 150  
console.log(discountAmount(0.3)); // 90  
console.log(discountAmount(0.2)); // 60
```

Simply we can say that we can done our **work 50%** before user requirement so that its can take **less time** when user want to access that.

For Example:- Getting data from **server** and perform further operations.



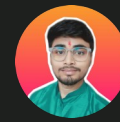
Currying Shorthand :-

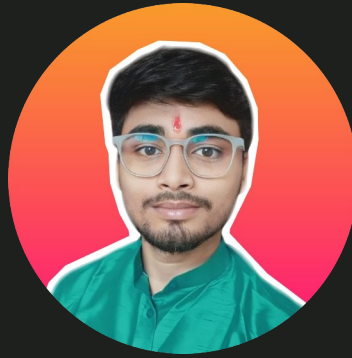
```
Currying  
  
const multiplication = a => b => c => a * b * c;  
  
console.log(multiplication(1)(2)(3)); // -> 6
```

Currying is an incredibly useful technique of **functional** programming which solves various purposes like passing **partial parameters** or avoiding unwanted repetitions like we tried with **product price** where price was passed **only once**.



 Save

 Ansh Mittal



Ansh Mittal

MERN | GenAI Dev

Follow For More

Save this pdf for reference

Do you like this post

let me know your thoughts

